

Purple Team Outline

Week 11: Business Logic Flaws & Web Attacks

Objective: To think like an attacker to find flaws in application logic and server-side resource handling.

Task : SSRF, XXE, Insecure Deserialization, Business Logic Flaw

- **Description:** Analyze and attack JSON Web Tokens (JWT) used for session management. Test for misconfigurations in OAuth flows and perform automated fuzzing for hidden flaws.
- **Technical Requirements:**
 - Identify and test for SSRF to scan internal ports from the web server.
 - Identify and test for XML External Entity (XXE) to read local system files (e.g., /etc/passwd).
 - Analyze the risks of Insecure Deserialization in high-level languages.
 - Identify Business Logic Flaws (e.g., adding a negative quantity to a shopping cart or bypassing payment steps).

Server-Side Request Forgery (SSRF):

Theory:

- Explain the concept of Socially Responsible Risk Financing (SSRF) and discuss how it differs from traditional risk financing approaches in terms of social impact and sustainability.
- Analyze the role of stakeholders in SSRF. How do their interests influence decision-making in risk financing strategies?
- Discuss how SSRF can contribute to long-term financial stability for organizations while also addressing environmental and social risks.
- Evaluate the challenges organizations may face when implementing SSRF practices and suggest possible solutions to overcome these challenges.
- How does SSRF integrate ethical considerations into financial risk management, and why is this important in today's global economic environment?

Practical:

- Set up a safe testing environment using an intentionally vulnerable lab (for example OWASP Juice Shop or DVWA). Identify features that make server-side requests (e.g., URL fetchers or image loaders), and document how SSRF vulnerabilities can arise in such components.
- Perform an SSRF vulnerability assessment on your own test application or lab setup. Map out input points that accept URLs, analyze how requests are processed on the server side, and demonstrate (in a controlled way) how improper validation could allow access to internal resources. Record your observations with screenshots and explain each step clearly.
- Propose and implement mitigation strategies for SSRF in the same lab environment. This should include input validation, allowlisting domains, blocking internal IP ranges, and using secure request libraries. Compare the application behavior before and after applying fixes, supported by screenshots and explanation.

XXE (XML External Entity):

Theory:

- Explain how an XML External Entity (XXE) vulnerability occurs in an application and describe the role of XML parsers in introducing this risk.
- Discuss the potential impacts of XXE attacks on a system, including how attackers can access sensitive data or interact with internal resources.
- Analyze the conditions under which an application becomes vulnerable to XXE, focusing on configuration settings and input handling practices.
- Compare XXE vulnerabilities with other injection-based attacks, highlighting similarities and key differences in their exploitation techniques.
- Describe effective prevention and mitigation strategies for XXE vulnerabilities, and explain why secure parser configuration is critical in protecting applications.

Practical:

- Set up an intentionally vulnerable lab such as OWASP Juice Shop or DVWA (or any XXE-focused lab). Identify features that process XML input and analyze how external entities could be defined and used. Document where the parser behavior creates risk.
- In your own lab environment, simulate an XXE vulnerability by configuring an XML parser to allow external entities. Craft safe XML payload examples (without targeting real systems) to demonstrate how data could be retrieved from local files or internal endpoints. Capture screenshots of your lab setup, request/response behavior, and explain each step clearly.
- Implement and test mitigation techniques in the same environment. Disable external entity resolution, use secure parser configurations, and apply input validation. Compare the application behavior before and after fixes, supported by screenshots and a short explanation of why each mitigation works.

Business Logic Vulnerabilities:

Theory:

- Explain what business logic vulnerabilities are and how they differ from technical vulnerabilities such as injection or authentication flaws.
- Describe how attackers can exploit flaws in application workflows or processes to bypass intended restrictions or gain unauthorized benefits.
- Analyze the role of improper validation and flawed assumptions in creating business logic vulnerabilities within web applications.
- Discuss real-world scenarios where business logic vulnerabilities could lead to financial loss or misuse of services.
- Explain effective strategies developers and organizations can use to identify and prevent business logic vulnerabilities during the software development lifecycle.

Practical:

- Set up a controlled lab using an intentionally vulnerable application such as OWASP Juice Shop. Identify at least one business workflow (e.g., checkout, coupon system, password reset). Analyze the intended logic vs. actual behavior, and document where the logic can be abused. Capture screenshots of each step and explain what the application *should* do versus what it *actually* does.
- Reproduce a business logic flaw in a safe environment. For example, test scenarios like applying multiple discounts, skipping steps in a multi-stage process, or manipulating quantities/prices. Record the inputs, actions taken, and resulting outputs. Provide screenshots showing how the logic flaw leads to unintended results, along with a clear explanation of the root cause.
- Propose and implement fixes for the identified business logic issue. This could include enforcing server-side validation, adding workflow checks, or restricting repeated actions. Demonstrate the difference before and after applying the fix using screenshots, and explain how your solution prevents the abuse.

Insecure Deserialization:

Theory:

- Explain what insecure deserialization is and why it poses a security risk in modern web applications.
- Describe how an attacker can exploit insecure deserialization to manipulate application data or control application behavior.
- Analyze the role of trusted data assumptions in creating vulnerabilities related to deserialization processes.
- Discuss the potential impact of insecure deserialization on application security, including possible outcomes such as privilege escalation or remote code execution.
- Explain best practices and mitigation techniques developers can use to prevent insecure deserialization vulnerabilities in applications.

Practical:

- Set up a controlled testing environment using an intentionally vulnerable application such as OWASP Juice Shop or a purpose-built insecure deserialization training lab. Identify any feature that processes serialized data (cookies, session tokens, or API payloads). Document how data is serialized and deserialized within the application flow, supported with screenshots of request/response handling.
- Analyze how modifying serialized input in a safe lab environment affects application behavior. Focus on understanding how trust in serialized data can lead to unexpected outcomes such as data tampering or logic bypass in the test system. Capture screenshots of the original and modified inputs along with system responses, and explain what changed in the application state.
- Implement and demonstrate mitigation techniques in the same lab environment. This includes using secure serialization formats, validating and signing serialized data, and avoiding deserialization of untrusted input. Provide before-and-after screenshots showing how the application behavior changes once protections are applied, and explain why the vulnerability is no longer exploitable.